

CONTAINER CRM

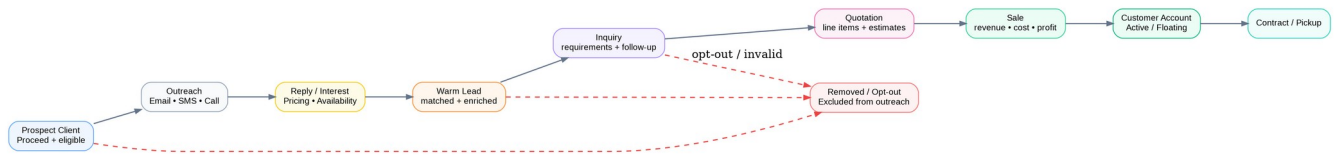
Project Details, Architecture, Schema & Flowcharts

Shipping Container Sales & Outreach CRM

React + Vite + TypeScript | Express + TypeScript | Supabase PostgreSQL/Auth/RLS

Prepared as an implementation and handover reference

August 2026



Document purpose

This document explains what the system is, how each module connects, how data should be stored, the critical business rules, the major database entities, API and frontend structure, security model, testing approach, deployment plan, current implementation status, and the next development steps. It is written so both technical and non-technical readers can follow the project.

Contents

- 1. Executive Summary
- 2. Project Goals and Business Context
- 3. Current Implementation Status
- 4. Technology Stack
- 5. System Architecture
- 6. Repository Structure
- 7. User Roles and Security
- 8. Core Data Model and ERD
- 9. Module-by-Module Functional Specification
- 10. Prospect-to-Sale Workflow
- 11. Removed & Deliverability Workflow
- 12. Database Schema Reference
- 13. API Design
- 14. Frontend Routes and UI Behavior
- 15. Calculations and Business Rules
- 16. Dashboards and Analytics
- 17. Import, Spreadsheet and Outreach Workflows
- 18. Environment Variables and Secrets
- 19. Local Development, Docker and Supabase
- 20. Testing and Quality Gates
- 21. Deployment Architecture
- 22. Implementation Roadmap
- 23. Risks, Controls and Non-Negotiables
- 24. Handover Checklist

1. Executive Summary

Container CRM is a production-oriented customer relationship management system designed for a shipping container sales operation serving the northern United States and Canada. It combines lead management, outreach eligibility, inquiries, quotations, sales, customer classification, contracts, pickup tracking, deliverability processing, spreadsheet-style data operations, and performance analytics in one connected system.

Core principle

The CRM is not a set of disconnected screens. Each module is a stage in one data pipeline. Company and contact identity should remain authoritative and reusable across Prospects, Warm Leads, Inquiries, Quotations, Sales, Customers, Contracts and outreach processing.

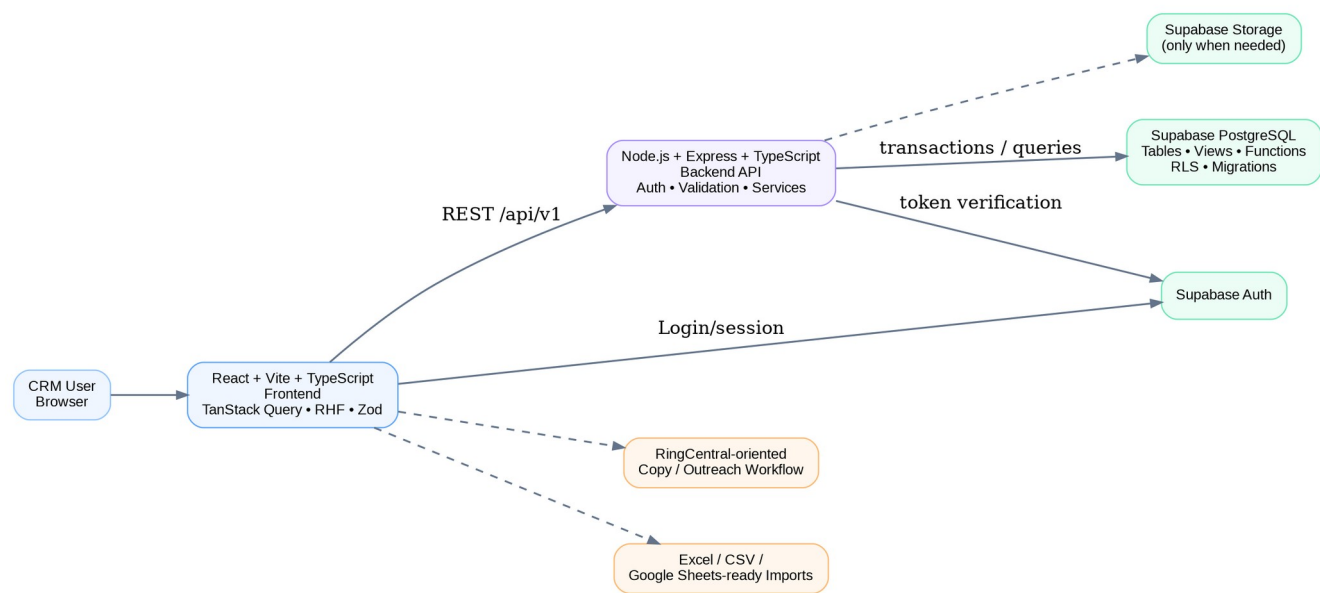


Figure 1. High-level application architecture

Primary objectives

- Centralize prospect, lead, inquiry, quotation and sales information.
- Prevent outreach to removed, opted-out, invalid or channel-restricted contacts.
- Provide spreadsheet-like workflows for teams already working with Excel or Google Sheets.
- Make sales and profit metrics authoritative and traceable to completed sales.
- Use Supabase PostgreSQL as the system of record, not localStorage or hardcoded frontend values.
- Keep the application secure using Supabase Auth, backend authorization and Row-Level Security (RLS).
- Support deployment with Vercel for frontend, Railway for backend and Supabase for database/auth.

What makes this CRM different

Area	Design decision
Identity	Companies and Contacts are shared authoritative records, not copied independently into each module.
Lead progression	Prospect -> Warm Lead -> Inquiry -> Quotation -> Sale is

	traceable and auditable.
Outreach	Phone, SMS and email eligibility are calculated centrally from removal and deliverability data.
Sales	Revenue, cost, gross profit and margin come from the Sales Tracker, not manual dashboard values.
Spreadsheet work	Bulk paste, staging, validation, duplicate review and transactional commit are first-class workflows.
Geography	Structured country/state/city/service territory data supports Northern USA and Canada.
Safety	Hosted Supabase is not modified until migrations pass local Docker validation.

2. Project Goals and Business Context

The organization needs to process large prospect lists, contact hundreds of potential customers, identify replies quickly, convert interested contacts into inquiries and quotations, track container sales, and understand performance by PIC, client, state/province, quantity, revenue and profit.

Key operating targets

Metric	Default / Business Rule
Monthly gross profit target	\$5,000
Daily email target	500
Daily call minimum	10
Daily preferred call target	15
Daily text target	Declared by PIC each day
Active customer	At least one qualifying completed purchase within the last 3 months
Floating customer	Has qualifying sales history, but latest qualifying purchase is older than 3 months

Geographic scope

The main operating territory is the northern United States and Canada. Coverage should be configured in structured service territories rather than embedded as free text in individual records.

3. Current Implementation Status

Current state - August 2026

Phase 4 is complete. Phase 5 foundation is implemented and the Prospect Clients -> Warm Leads -> Inquiry foundation has started. All currently available non-Docker validation gates pass. Hosted Supabase has not been linked or modified.

Area	Status
Express backend foundation	Complete
Environment validation / Supabase and PostgreSQL clients	Complete
Authentication / role authorization	Complete
Request validation, logging, CORS, Helmet, rate limiting	Complete
Shared domain schemas and utilities	Complete foundation
Migrations 001-021	Created
Migration 022 Phase 5 hardening	Created, not yet executed against PostgreSQL
PIC / locations / territories / companies / contacts / catalog APIs	Implemented
Prospect / Warm Lead / Inquiry pages and basic APIs	Started / scaffolded
Frontend reusable tables/forms/dialogs/states	Implemented foundation
Automated tests	26 passing (8 backend + 18 shared)
Frontend tests	Not yet added at this checkpoint
Lint / TypeScript / builds	Passing
Docker local Supabase validation	Pending due WSL/C: space issue
Hosted Supabase db push	Not performed

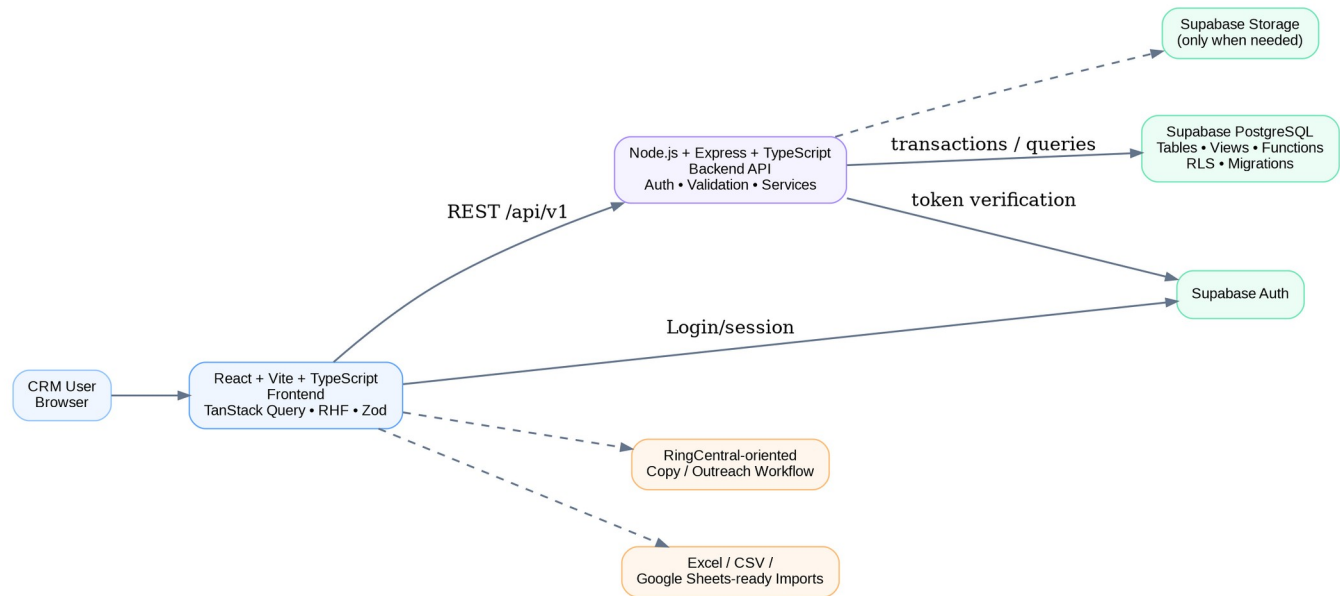
Current local blocker

Docker Desktop has been installed, but WSL update/repair currently fails because the Windows C: drive has only about 300 MB free. This blocks local Supabase startup. The project itself does not need to be moved. The safe next infrastructure task is to free several GB of system-drive space, complete WSL, confirm Docker Engine is running, then execute local Supabase validation.

4. Technology Stack

Layer	Technology	Purpose
Frontend	React + Vite + TypeScript	Browser application and user interface
Routing	React Router	Module navigation and protected routes
Server state	TanStack Query	API fetching, cache, invalidation
Forms	React Hook Form + Zod	Typed form handling and validation
Grid	AG Grid Community / suitable production data grid	Spreadsheet-style prospect/outreach/import screens
Charts	Recharts	Dashboard and analytics visuals
Files	SheetJS + Papa Parse	Excel/CSV import/export
Backend	Node.js + Express + TypeScript	Secure business API and workflows
Database	Supabase PostgreSQL	System of record
Authentication	Supabase Auth	User identity and sessions
Authorization	Backend roles + RLS	Data/action access control
Frontend hosting	Vercel	Production web deployment
Backend hosting	Railway	Express API deployment
Source control	GitHub	Version control and CI
Local DB runtime	Docker + Supabase CLI	Safe migration/RLS/integration testing

5. System Architecture



Request flow

1. The user interacts with the React frontend.
2. React calls /api/v1 endpoints through the Express backend for protected business operations.
3. The frontend may use Supabase Auth with the publishable key for sign-in/session handling.
4. The backend verifies identity and authorization before executing business services.
5. Repository methods read/write PostgreSQL using allowlisted queries and transactions.
6. RLS provides an additional database protection layer for browser-accessible data.
7. Domain events preserve important state transitions for audit and activity timelines.

6. Repository Structure

```
container-crm/
├── frontend/    React + Vite + TypeScript
│   ├── src/app/
│   ├── src/components/
│   ├── src/features/
│   ├── src/lib/
│   └── .env.local
├── backend/    Express + TypeScript
│   ├── src/controllers/
│   ├── src/routes/
│   ├── src/services/
│   ├── src/repositories/
│   ├── src/middleware/
│   ├── src/config/
│   └── .env
├── shared/     Shared types, Zod schemas, enums and utilities
├── supabase/   config, migrations, seed, database tests
├── docs/       architecture and validation records
├── package.json workspace scripts
└── .gitignore
```

The root acts as a lightweight workspace. Frontend-specific dependencies stay in frontend, backend dependencies stay in backend, and cross-layer contracts belong in shared. This reduces drift between API validation and frontend form validation.

7. User Roles and Security

The application uses Supabase Auth for identity and backend authorization for operational permissions. RLS is the database safety layer, while Express middleware protects API actions.

Security control	Responsibility
Supabase Auth	Sign-in, session and authenticated user identity
Profile / PIC mapping	Maps authenticated users to CRM role and PIC context
Backend authentication	Validates the access token for /api/v1
Role authorization	Controls admin/manager/PIC sensitive actions
Zod validation	Rejects invalid payloads before business logic
Transactions	Ensures multi-step workflows commit or roll back together
RLS	Restricts direct browser database access by role/user context
Secret separation	Service key and DATABASE_URL remain backend-only
Structured/redacted logging	Avoids leaking passwords, tokens and secret keys

Security rule
Anything beginning with VITE_ is considered browser-visible. Never put SUPABASE_SECRET_KEY, service-role key, DATABASE_URL, database password or third-party secrets in the frontend environment file.

8. Core Data Model and ERD

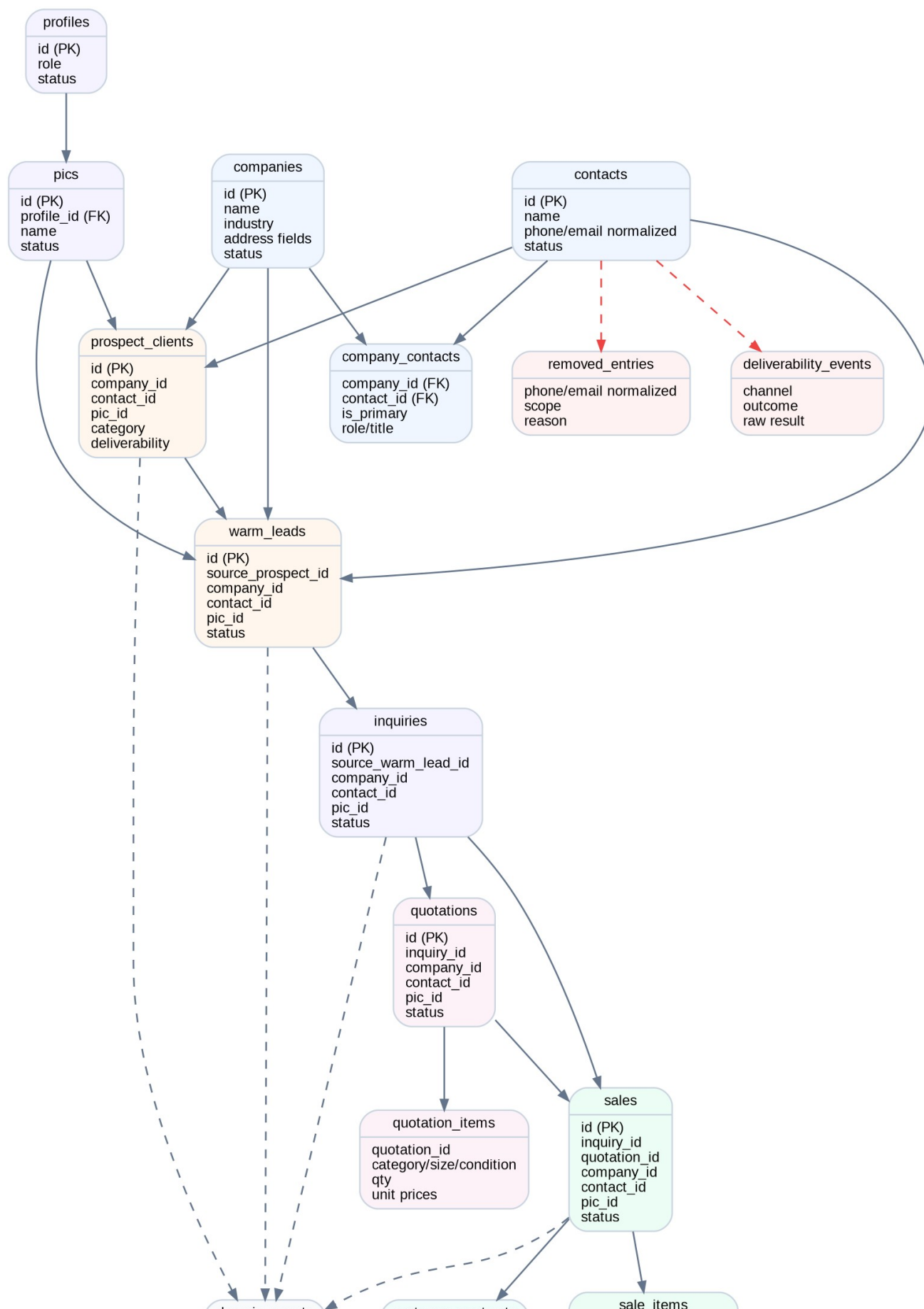


Figure 2. Simplified high-level entity relationship diagram

Data ownership model

Company and Contact information should be stored once and referenced by IDs. Pipeline tables store workflow-specific fields and foreign keys, while sales and historical records may preserve snapshots where appropriate for audit or operational accuracy.

Domain	Authoritative data
Identity	profiles, pics
Geography	countries, areas/states/provinces, cities, service_territories
Party master data	companies, contacts, company_contacts
Catalog	categories, sizes, conditions, availability
Pipeline	prospect_clients, warm_leads, inquiries
Commercial	quotations, quotation_items, sales, sale_items
Customer lifecycle	customer views/status, contracts, pickups
Outreach control	removed_entries, deliverability_events, eligibility views/services
Operations	daily_completed_tasks, import_batches, import_rows
Audit/analytics	domain_events, activity history, analytics views

9. Module-by-Module Functional Specification

Module	Purpose
PIC Management	Manage operational PICs, active/inactive lifecycle, profile linkage and reporting ownership.
Locations & Territories	Countries, states/provinces, cities and structured service coverage for USA/Canada.
Companies	Authoritative company record including normalized name, industry and structured address.
Contacts	Authoritative person/contact record with normalized phones/emails and duplicate candidates.
Company-Contact Relationships	Connect contacts to companies, support role/title and one primary contact per company.
Container Catalog	Category, size, condition and location-based availability

	with quantity/status.
Prospect Clients	Large spreadsheet-style list of contacts that have not replied or opted out and remain eligible.
Warm Leads	Contacts who replied, requested pricing/availability/quotation or expressed genuine interest.
Inquiries	Detailed customer requirements, container needs, timing, logistics, follow-up and status.
Quotations	Line-item commercial proposal with estimated buy/sell values and gross profit.
Sales Tracker	Authoritative completed transaction source for revenue, cost, profit, quantity and customer status.
Customer Accounts	Derived customer view including Active/Floating classification and sales history.
Contracts & Pickups	Post-sale contractual records, scheduled pickups and overdue detection.
Daily Completed Tasks	Daily PIC targets and completed emails/calls/texts plus generated leads/inquiries/quotes/sales.
Removed Sheet	Manual/bulk channel or entity removal with propagation and eligibility recomputation.
Deliverability	Email/SMS/phone outcome history and mapped final channel availability.
Contact Outreach Sheet	Filtered, eligible RingCentral-ready copying/export by channel.
Dashboards & Analytics	Profit, pipeline, outreach, state/province, PIC, best clients, losses and operational alerts.

Prospect Clients - A to Q structure

Column	Field
A	Date Added
B	PIC
C	Category: Proceed / Removed
D	SMS Deliverability
E	Email Deliverability

F	Industry
G	Service Locations
H	Country
I	State / Province
J	City
K	Company Name
L	Contact Person
M	Contact Number - Direct Line
N	Contact Number 2
O	Email - Active
P	Email 2
Q	Address

Optional helper fields R/S/T/U are Street, City Helper, State/Province Helper and Country Helper. S -> J, T -> I, U -> H, and R+S+T+U generate Q Address.

10. Prospect-to-Sale Workflow

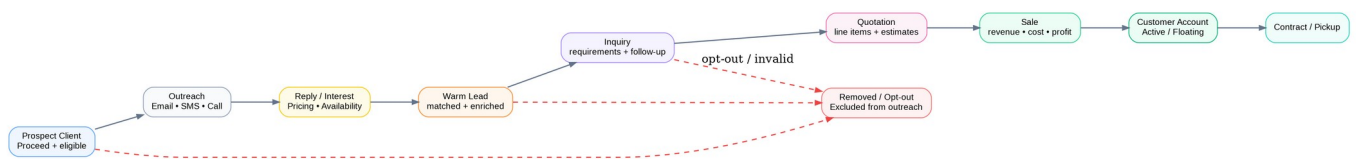


Figure 3. Primary CRM progression

Prospect definition

A Prospect Client has not replied, has not requested a quotation, has not actively expressed interest and has not opted out. Eligible prospects can continue to receive outreach according to their channel status.

Warm Lead definition

A Warm Lead has replied through email, SMS, phone/RingCentral or another supported channel; asked about container availability/specifications/pricing; requested a quotation; or otherwise expressed genuine interest.

Conversion requirements

- Normalize phone and email before matching.
- Search existing Prospects, Warm Leads, Contacts, Customers, Inquiries, Quotations, Sales, Removed records and deliverability history.
- Show candidate matches and require explicit selection if ambiguous.

- Use a dedicated transactional Prospect -> Warm Lead conversion operation.
- Preserve the source Prospect; never delete it as part of conversion.
- Create domain events/activity records.
- Prevent duplicate open Warm Leads and repeated-submit duplicate conversion.
- Allow optional Inquiry creation inside the same controlled workflow.

Inquiry status model

New Inquiry -> Under Review -> Information Requested -> Quotation Required -> Quotation Prepared -> Quotation Sent -> Awaiting Response -> Negotiating -> Converted to Sale / Closed / Lost / Spam or Invalid.

Quotation status model

Draft -> Sent -> Viewed -> Accepted / Rejected / Cancelled -> Converted. Discounts, taxes, approval workflow, version history and expiration tracking are intentionally out of scope.

11. Removed & Deliverability Workflow

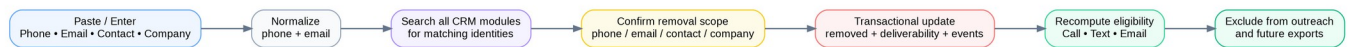


Figure 4. Removal and outreach eligibility flow

Removal scopes

Phone Only | Email Only | Phone and Email | Entire Contact | Entire Company

A phone opt-out must not automatically disable a valid email unless the confirmed removal scope applies to the full contact/company. The inverse rule applies to email unsubscribe versus phone contact.

Deliverability mapping examples

Observed result	Mapped status / effect
Landline	Calls Only
SMS undeliverable but calls work	Calls Only
Text works but calls prohibited	Text Only
Invalid / disconnected / wrong number	Removed
Opted Out	Removed
Call and SMS work	Call/Text
Hard Bounce / Recipient Not Found / Invalid Format	Email Removed
Unsubscribed / Spam Complaint	Email Removed
Soft Bounce / Mailbox Full / Blocked	Mail Delivery Report with warning

Central eligibility rules

Channel	Eligibility
Call	Category=Proceed; SMS status Call/Text or Calls Only; normalized valid number; number/contact/company not removed.
Text	Category=Proceed; SMS status Call/Text or Text Only; normalized valid number; number/contact/company not removed.
Email	Category=Proceed; active email exists; deliverability acceptable; email/contact/company not removed.

12. Database Schema Reference

The migration set currently spans 001-022. The schema below is a conceptual reference; exact migration columns remain the implementation source of truth.

Entity / group	Purpose
profiles	Auth-linked application profile, role, status.
pics	PIC identity and profile linkage.
countries / areas / cities	Structured geography.
service_territories	Configured sales/service coverage.
companies	Authoritative company master.
contacts	Authoritative contact master with normalized contact channels.
company_contacts	Many-to-many party relation with primary-contact rule.
container_categories	Dry, High-Cube, Double-Door, Open-Top, Flat-Rack, Refrigerated, etc.
container_sizes	10ft, 20ft, 20ft HC, 40ft HC, 45ft HC, 53ft HC.
container_conditions	Brand New, One Trip, Cargo Worthy, WWT, As-Is, Refurbished, Modified, Used.
container_availability	Category/size/condition/location quantity and availability status.
prospect_clients	Prospecting stage plus PIC, deliverability and source data.
warm_leads	Interested/replied lead, source prospect, reply context,

	status.
inquiries	Customer requirement, container specification, logistics, follow-up.
quotations	Quotation header, source inquiry, customer/contact/PIC/status.
quotation_items	Container line items and estimated prices.
sales	Sale header and source inquiry/quotation.
sale_items	Quantity, buying/selling price, totals, profit, margin.
customer_contracts	Contract linked to sale/company.
pickups	Scheduled pickup lifecycle and overdue state.
daily_completed_tasks	PIC daily target/actual metrics.
removed_entries	Normalized channel/entity removal and scope.
deliverability_events	Raw deliverability/contact channel outcome history.
import_batches / import_rows	Staged transactional bulk processing.
domain_events	Auditable event stream for key workflow changes.
analytics views	Authoritative dashboard aggregations.

Key relationship rules

- Company-to-Contact is many-to-many, with no duplicate relationship rows.
- Only one contact should be primary for a company at a time.
- Pipeline entities reference company/contact/PIC rather than only saving names.
- Quotation and Sale line items are child tables to support multiple containers per transaction.
- Customer status is derived from qualifying Sales, not manually maintained as the primary source.
- Removal and deliverability are historical/event-oriented; do not overwrite away the original processing history.

13. API Design

All operational endpoints are versioned under /api/v1 and protected by authentication unless explicitly documented otherwise.

Route group	Purpose
/pics	PIC list/create/update/deactivation
/locations/countries	Country reference data

/locations/areas	State/province data
/locations/cities	City data
/territories	Service territory management
/companies	Company CRUD/search/filter/pagination
/companies/duplicates	Duplicate candidate detection
/contacts	Contact CRUD/search/filter/pagination
/contacts/duplicates	Phone/email duplicate candidates
/company-contacts	Relationship management
/catalog/categories	Container category setup
/catalog/sizes	Container sizes
/catalog/conditions	Container conditions
/catalog/availability	Location/category/size/condition availability
/prospects	Prospect clients
/warm-leads	Warm leads
/inquiries	Inquiry foundation

Planned transition endpoints

POST /api/v1/prospects/:id/convert-to-warm-lead
 POST /api/v1/warm-leads/match
 POST /api/v1/warm-leads/:id/enrich
 POST /api/v1/warm-leads/:id/create-inquiry

API response contract

Success
 { "success": true, "data": {...}, "meta": {...}, "requestId": "uuid" }

Failure
 { "success": false, "error": { "code": "...", "message": "...", "details": [] }, "requestId": "uuid" }

14. Frontend Routes and UI Behavior

Route	Purpose
/login	Authentication
/dashboard	Main executive/operational dashboard
/dashboard/outreach	Outreach performance
/dashboard/inquiries	Inquiry funnel
/prospects	Prospect spreadsheet/list
/warm-leads	Warm lead list/enrichment
/inquiries	Inquiry management
/quotations	Quotation list/builder
/sales	Sales tracker
/customers	Customer accounts
/customers/active	Active customers
/customers/floating	Floating customers
/outreach	Contact Outreach Sheet
/daily-tasks	Daily Completed Tasks
/contracts	Contracts
/pickups/overdue	Overdue pickup tracking
/catalog/*	Container setup and availability
/data/removed	Removed Sheet
/data/deliverability	Deliverability processing
/data/imports	Bulk import processing
/analytics/*	PIC, best clients, state/province, profit and loss analytics
/settings/*	Targets, territories, container and system settings

UI principles

- Use screenshots as primary visual reference.

- Enterprise layout with sidebar, top navigation, clean cards, status badges and spreadsheet modules.
- Every module must show loading, empty, error and success/feedback states.
- Forms use React Hook Form + shared Zod schemas.
- Lists support search, allowlisted sorting, filters and pagination/virtualization.
- A module is not considered complete until its UI calls real API functions; localStorage is not the application database.
- Route-level lazy loading should be added to reduce the current Vite main bundle warning.

15. Calculations and Business Rules

Sales calculations

Metric	Formula
Total Buying Price	Quantity x Buying Price per Unit
Total Selling Price	Quantity x Selling Price per Unit
Gross Profit	Total Selling Price - Total Buying Price
Profit Margin	Gross Profit / Total Selling Price x 100; zero-safe when total selling price is 0

Currency values must use PostgreSQL NUMERIC/DECIMAL or equivalent exact arithmetic, never floating point as the source of truth.

Customer classification

Active: at least one qualifying completed purchase within the most recent 3 months. Floating: has qualifying purchase history but the latest qualifying purchase is older than 3 months. A company with no qualifying sales history is neither Active nor Floating by this rule.

Phone normalization

Accept inputs such as 6125550182, +1 612 555 0182, 1-612-555-0182, (612) 555-0182 and 612.555.0182. Match using the normalized value while displaying US/Canada numbers as (xxx) xxx-xxxx.

16. Dashboards and Analytics

Metric / visual	Authoritative source
Monthly Gross Profit	Sales Tracker qualifying statuses and sale items
Profit Target Progress	\$5,000 default monthly target vs actual gross profit
Revenue / Buying Cost / Margin	Authoritative Sales Tracker totals
Daily Outreach	Daily Completed Tasks actuals vs targets
Pipeline counts	Prospects, Warm Leads, Inquiries, Quotations, Sales

Eligible Outreach	Central eligibility service/view
Proceed Prospects by State	Prospects where Category=Proceed grouped by country/state/province
Active Clients by State	Active customer companies joined to completed sales and location
Best Clients by Quantity	Qualifying total units purchased; sortable by quantity/revenue/profit/sales count
PIC Performance	Outreach, lead, inquiry, quote, sales, quantity, revenue and profit by PIC
Inquiry Funnel	Status and source conversion
Loss Reasons	Lost inquiry/sale reasons
Special Arrangements	Frequency and conversion outcome
Overdue Pickups	Scheduled past due and unresolved pickup statuses

Analytics requirement

Every metric must document its source tables, qualifying statuses, date field, grouping rules, exclusions, null handling and timezone behavior. Dashboard values must never be hardcoded.

17. Import, Spreadsheet and Outreach Workflows

Bulk paste/import engine

The import engine should recognize 17 columns (A-Q) or 21 columns (A-U), stage rows first, normalize and validate them, identify duplicate/removal conflicts, show a preview, then commit transactionally.

Supported modes: Append as New Rows (default), Update Matching Records, Paste into Selected Cells, Replace Existing Values, Fill Empty Cells Only.

Contact Outreach Sheet

This spreadsheet-style screen prepares eligible contacts for RingCentral or other manual outreach workflows. It supports selecting filtered rows and copying phone numbers, emails, names+numbers, names+emails or full rows in one-per-line, comma-separated, semicolon-separated or tab-separated formats.

Before copying/exporting, the UI should show eligible count plus exclusions: removed, invalid, bounced, channel-restricted and incomplete records.

18. Environment Variables and Secrets

Frontend: frontend/.env.local

```
VITE_SUPABASE_URL=https://YOUR_PROJECT_REF.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY=sb_publishable_...
VITE_API_BASE_URL=http://localhost:3001/api
```

Backend: backend/.env

```
PORT=3001
NODE_ENV=development
CORS_ORIGINS=http://localhost:5173
SUPABASE_URL=https://YOUR_PROJECT_REF.supabase.co
SUPABASE_PUBLISHABLE_KEY=sb_publishable_...
SUPABASE_SECRET_KEY=sb_secret_...
DATABASE_URL=postgresql://...
```

Never commit secrets

Keep real .env files ignored by Git. Frontend only receives public values. Backend-only secrets include SUPABASE_SECRET_KEY/service role, DATABASE_URL, database password and third-party API secrets.

19. Local Development, Docker and Supabase

Docker Desktop is used only to run a safe local Supabase environment. It does not replace Windows and it does not move the application to Linux. On Windows, Docker commonly uses WSL 2 in the background.

Local validation sequence

1. Confirm Docker Engine is running
docker info
2. Go to project
cd D:\CRM\container-crm
3. Verify Supabase CLI
npx supabase --version
4. Start local Supabase
npx supabase start
5. Rebuild local database from migrations
npx supabase db reset
6. Run DB/RLS/integration tests
7. Generate database TypeScript types
8. Only after approval: link hosted project and push migrations

Current infrastructure issue

WSL repair/update currently reports insufficient space on C:. The CRM project can remain where it is. The operating system needs several GB of free C: space for WSL. After WSL succeeds, Docker data can be configured to use D: for its large disk image.

20. Testing and Quality Gates

Test layer	Examples
Shared/unit	Phone/email normalization, address generation, exact calculations, territory matching, schemas
Service	Company/contact duplicates, relationship uniqueness, availability rules, conversion eligibility
Repository integration	PostgreSQL CRUD, constraints, sorting/filtering, transactions
Database	Migration reset, pgTAP constraints/functions, indexes
RLS	Role-by-role allowed/denied operations
API integration	Authenticated endpoint behavior and error contracts
Frontend component	Forms, dialogs, loading/empty/error states, conversion UX
End-to-end	Prospect -> Warm Lead -> Inquiry -> Quotation -> Sale -> Customer -> Contract/Pickup

Current verified gates

npm ls --all: passed; invalid dependencies: []; extraneous dependencies: []; ESLint: passed; TypeScript: passed; tests: 26 passed; frontend and backend builds: passed. Docker-dependent database and RLS execution remains pending.

21. Deployment Architecture

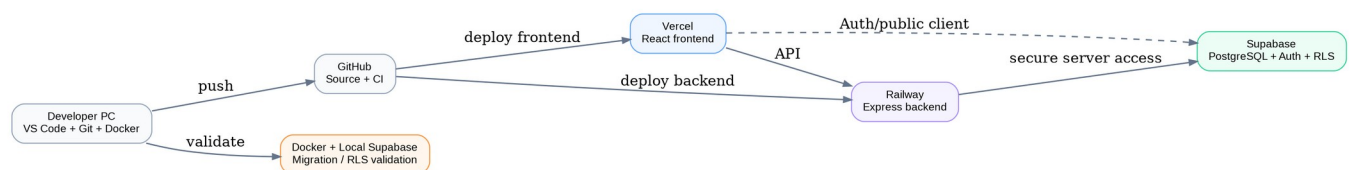


Figure 5. Planned development and production deployment

Deployment responsibilities

Platform	Responsibility
GitHub	Repository, branches, pull requests, CI source

Vercel	React frontend build and hosting
Railway	Node/Express backend runtime and secret environment variables
Supabase	PostgreSQL, Auth, RLS and optional Storage
Docker on developer PC	Local Supabase validation before hosted migration changes

22. Implementation Roadmap

Phase	Scope	Status
Phase 3	Repository consolidation + Supabase foundation	Completed foundation
Phase 4	Express backend foundation, security, migrations	Complete
Phase 5	Shared master data: PIC, locations, companies, contacts, catalog; initial pipeline pages	Implemented
Phase 6	Prospect -> Warm Lead -> Inquiry transitions, audit events, frontend tests, lazy loading	Next core application phase
Phase 7	Inquiry hardening -> Quotation Builder -> Quotation acceptance -> Sale conversion	Planned
Phase 8	Sales Tracker -> Customer Accounts -> Active/Floating -> best clients	Planned
Phase 9	Contracts, pickups, daily tasks, outreach, removed and deliverability	Planned
Phase 10	Dashboards, analytics, performance, final UI polish, E2E, deployment	Planned

Immediate next technical steps

- Free enough C: space for WSL update/repair; do not move the large project just for this.
- Confirm Docker Engine is running with docker info.
- Run `npx supabase start` and `npx supabase db reset`.
- Fix migration 001-022 execution errors until a clean reset passes.
- Run PostgreSQL repository, transaction, constraint and RLS tests.
- Generate database TypeScript types.
- Then complete Phase 6 Prospect -> Warm Lead -> Inquiry workflow and frontend tests.

23. Risks, Controls and Non-Negotiables

Risk	Control
Accidental outreach to opted-out contact	Central eligibility service + removed/deliverability history + exclusion before copy/export
Duplicate customer/lead records	Normalized matching + candidate review + no silent merge
Inconsistent metrics	Sales Tracker and database analytics views are authoritative
Partial conversion/import writes	Use database transactions and rollback
Frontend exposing secrets	Only public VITE_ values; backend secrets never browser-visible
Migration breaks hosted DB	Local Docker reset and tests before hosted db push
Business rules duplicated across pages	Shared schemas/utilities + backend services + authoritative views
Currency rounding errors	Exact numeric/decimal types; server/database verification
UI looks complete but actions are fake	Vertical-slice rule: UI + API + service + repository + tests
Loss of history	Domain events, source foreign keys and event-oriented deliverability/removal records

Explicit non-negotiables

- No localStorage as the production database.
- No hardcoded dashboard metrics.
- No service-role/secret key in the frontend.
- No floating-point currency as the source of truth.
- No silent duplicate merges.
- No deleting the source Prospect when converting it.
- No hosted supabase db push before local migration validation.
- No module is complete until its UI is functionally connected to the API.

24. Handover Checklist

- ☐ Repository installs reproducibly with npm ci.
- ☐ Frontend, backend and shared typecheck/lint/build pass.
- ☐ Docker Engine runs and local Supabase starts.
- ☐ All migrations apply from a clean npx supabase db reset.
- ☐ Seed/reference data applies successfully.
- ☐ RLS tests pass for all roles.

- ☐ Database types are generated and consumed where appropriate.
- ☐ Prospect -> Warm Lead -> Inquiry transaction is tested.
- ☐ Quotation -> Sale conversion is transactional and idempotent.
- ☐ Sales calculations are authoritative and exact.
- ☐ Active/Floating customer logic is verified against sale dates.
- ☐ Removed and deliverability propagation is tested.
- ☐ Outreach copy/export excludes ineligible contacts.
- ☐ Dashboards document their source tables and status/date rules.
- ☐ Frontend component and E2E tests cover critical workflows.
- ☐ Production environment variables are configured in Vercel/Railway without secrets in Git.
- ☐ Hosted Supabase migrations are reviewed before first push.
- ☐ Deployment smoke test confirms auth, API, database and core workflow.

Final project definition

Container CRM is a connected sales and outreach operating system: structured prospecting feeds qualified warm leads and inquiries; quotations feed sales; sales drive customer status and profitability; removal and deliverability rules protect outreach; analytics summarize authoritative operational data. The architecture is intentionally migration-driven, typed, transactional and testable.